

ProcessTensors.jl: An ITensor-native framework for non-Markovian open quantum dynamics

Gauthameshwar S. ^{1,2,*}, Aaron Sander ³, John F. Kam ^{4,5}, Dario Poletti ^{1,2,6}, and Kavan Modi ^{1,2,†}

¹ Singapore University of Technology and Design, Singapore

² Centre for Quantum Technologies, Singapore

³ Technical University of Munich, Munich, Germany

⁴ Monash University, Melbourne, Australia

⁵ Agency for Science, Technology and Research (A*STAR), Singapore

⁶ MajuLab, CNRS-UNS-NUS-NTU International Joint Research Unit, Singapore

* gauthameshwar_s@mymail.sutd.edu.sg, † kavan_modi@sutd.edu.sg

Abstract

We present ProcessTensors.jl, a Julia package for learning, constructing, and using process tensors with ITensor. The paper introduces process tensors from the ground up and connects the equations to tensor diagrams, tagged ITensor indices, and working code. Through guided examples, users can build process tensors, apply different sequences of quantum instruments, use testers with ancillary systems, and inspect memory across the temporal bonds. We implement exact and compressed ACE constructions and benchmark them against established software. We also review the current process-tensor software landscape and explain how the package can be extended with further PT-MPO construction methods, including ideas based on TEMPO, TEDOPA, and periodic process tensors. Our goal is to make process tensors approachable for new users while providing a clear foundation for future development of this package.

Copyright attribution to authors.

This work is a submission to SciPost Physics Codebases.

License information to appear upon publication.

Publication information to appear upon publication.

Received Date

Accepted Date

Published Date

Markers indicate whether a subsection is primarily theoretical () , focuses on Julia walkthroughs () , or both () .

Contents

1	Introduction	2
2	Liouville-space and tensor-network foundations	5
2.1	  A graphical dictionary for tensors	5
2.2	  From operator space to Liouville space	6
2.3	  Left and right actions	8
2.4	  Vectorised many-body MPS and Liouvillian MPOs	10
3	Process tensor theory and construction	13
3.1	 From quantum channels to multi-time processes	13
3.2	 Choi representation, instruments, and process contraction	16

3.3	  Anatomy of a ProcessTensor	16
3.4	 Ingredients of a process-tensor construction in Julia	17
3.5	 Building and inspecting the process tensor	19
4	Quantum instruments	23
4.1	 Operational meaning	23
4.2	  Instrument organisation in <code>ProcessTensors.jl</code>	23
4.3	  InstrumentSeq: a temporal analogue of <code>OpSum</code>	23
5	Contracting and evaluating processes	23
5.1	  General contraction with <code>evaluate_process</code>	23
5.2	  Reduced dynamics with <code>evolve</code>	23
5.3	 Open process legs	23
6	Common process-tensor workflows	23
6.1	 Final reduced density matrices	23
6.2	 Observable expectation values	23
6.3	  Probabilities and trace-non-preserving branches	23
6.4	  Multi-time correlation functions	23
7	Testers and enlarged interventions	23
8	Characterising process tensors and quantum memory	23
9	Validation and ACE benchmarks	23
9.1	  Correctness against dense and exact references	23
9.2	  Compression accuracy and temporal bond dimension	23
9.3	 Runtime and memory scaling	23
9.4	  Comparison with the C++ ACE toolkit	23
10	Conclusion and outlook	23
A	 How to read the Julia listings	24
B	  Time dynamics in Liouville space	25
	References	26

1 Introduction

In principle, describing an open quantum system is easy: write down a giant wavefunction for the entire universe, place it in an even more gigantic Hilbert space, and evolve it according to the Schrödinger equation. Somewhere inside that wavefunction lies everything one could wish to know about the system of interest. In practice, of course, we cannot keep track of every surrounding particle, lattice vibration, stray electromagnetic field, or experimental imperfection and calculate their joint correlated dynamics. We therefore do what physicists have always done when the universe becomes inconveniently large: draw a boundary. The degrees of freedom we wish to study are called the *system*; everything beyond this boundary is called

the *environment* or *bath*. Open quantum-system theory asks how the bath affects the system without requiring us to simulate the bath in all its microscopic detail.

The most familiar answers we would see in this regard are those that rely on weak-coupling, short-memory, or related approximations on the baths that lead to time-local master equations. In the Markovian limit, the system does not need a record of its earlier history in order to determine what happens next. Markovian master equations provide an extremely successful workhorse for open quantum dynamics, with the Gorini–Kossakowski–Sudarshan–Lindblad form supplying the standard generator of quantum dynamical semigroups [1, 2]. Nevertheless, weak coupling and a clean separation of timescales are not available in every problem. Structured reservoirs, strong system–bath coupling, slow environmental modes, and delayed feedback can all preserve information about earlier dynamics [3, 4]. In these regimes, memory is part of the physics rather than a correction that can safely be ignored.

The difficulty is not merely that the environment remembers, but that the system may give us no indication of *what* it remembers if we trace out the bath. To see this, consider two joint system–environment states at some time t_1 ,

$$\rho_{SE}^{(1)}(t_1) \quad \text{and} \quad \rho_{SE}^{(2)}(t_1), \quad (1)$$

which reduce to exactly the same system state,

$$\text{Tr}_E[\rho_{SE}^{(1)}(t_1)] = \text{Tr}_E[\rho_{SE}^{(2)}(t_1)] = \rho_S(t_1). \quad (2)$$

As far as the system density matrix is concerned, these two situations are indistinguishable. The environment, however, may disagree. It can retain different states or different correlations with the system in the two cases. Consequently, evolving both joint states with the same system–environment unitary need not produce the same reduced state at a later time:

$$\text{Tr}_E[U_{2:1}\rho_{SE}^{(1)}(t_1)U_{2:1}^\dagger] \neq \text{Tr}_E[U_{2:1}\rho_{SE}^{(2)}(t_1)U_{2:1}^\dagger]. \quad (3)$$

Thus, even perfect knowledge of the present system state need not be sufficient to predict its future. The information distinguishing the two experiments has simply moved somewhere beyond the boundary we drew earlier.

The limitation becomes sharper when we intervene our system along the way. Imagine performing two experiments with different histories and, at an intermediate time t_1 , discarding the system state and preparing the same fresh state P in both. This operation—often called a *causal break*—removes any information that could pass directly from the past to the future through the system. Immediately afterward, both experiments therefore contain the same reduced state,

$$\rho_S^{(1)}(t_1^+) = \rho_S^{(2)}(t_1^+) = P. \quad (4)$$

Nevertheless, the intervention acts on a system that may already be correlated with its environment. A measurement outcome, for example, can condition the environment into a corresponding state, while repreparing P makes the system look identical in every run. If the later dynamics still depends on the earlier preparation or measurement outcome, then the memory must have travelled across the causal break through the environment. Such memory can remain hidden in an uninterrupted reduced trajectory, and can even occur when familiar two-time criteria describe the unperturbed evolution as Markovian [5].

This leaves a state-to-state description with an awkward question. What map should take P at t_1 to the state at t_2 , if the same P can lead to different futures depending on what happened before it was prepared? There is no contradiction in quantum mechanics here; rather, we have asked the reduced state to remember information that it no longer contains. The missing input is the history of interventions—the preparations, controls, and measurements through which

the system and its environment arrived at the present experiment. A complete description of the quantum process must therefore accept an entire ordered sequence of operations, rather than only a density matrix at one particular time.

The process tensor provides precisely such a description [6–8]. For a sequence of interventions $(\mathcal{A}_0, \mathcal{A}_1, \dots, \mathcal{A}_{n-1})$, it returns the final system state,

$$\rho_S(t_n) = \mathcal{T}_{n:0}[\mathcal{A}_{n-1}, \dots, \mathcal{A}_0], \quad (5)$$

or, when the interventions contain measurements, the corresponding outcome probabilities and conditional states. A reduced trajectory tells us what happened under one prescribed history; a process tensor tells us what could have happened had we intervened differently along the way. It does not reveal every microscopic bath degree of freedom, but retains their complete influence on experiments performed through the system (the mathematical construction behind this statement—including its Choi representation and temporal input and output spaces—will be developed carefully in Sec. ??). For the present discussion, its operational meaning is enough). This makes causal breaks, multi-time correlations, instrument sequences, Markov order, and correlations across temporal partitions accessible within one description. In the Markovian limit, the process separates into independent transformations between successive times; otherwise, its temporal parts remain correlated, recording how information stored in the environment can return later. Memory thus becomes part of the object being manipulated rather than a feature inferred from a single trajectory.

The generality of a process tensor comes at an exponential cost: every additional intervention time introduces another pair of system indices, thus causing the exponential growth of the representation space of a process tensor, similar to how a many-body wavefunction space grows with the number of particles. Tensor networks make this manageable by representing the process as a matrix product operator in time, or PT-MPO [9]. The open legs provide the slots for system operations, while the internal bonds carry correlations between different parts of the process. When these temporal correlations are compressible, singular-value truncations reduce the representation to a controllable bond dimension. Different algorithms construct this PT-MPO by exploiting different structures of the environment: PT-TEMPO contracts the influence functional of Gaussian bosonic baths [9, 10], ACE combines and compresses the influence of independent bath modes [11], and time-translation-invariant approaches target long-time processes using repeating tensors [12]. The process tensor is therefore not made small by assumption; it is made useful whenever the environment stores its relevant memory in a compressible temporal space.

The principal open-source packages that construct reusable PT-MPOs are OQuPy, which implements PT-TEMPO for Gaussian bosonic environments [13], and the C++ ACE toolkit, which supports ACE, PT-TEMPO, and related contraction schemes for a broader class of independent bath modes [14]. Packages such as `QuantumDynamics.jl` and `MPSDynamics.jl` provide complementary influence-functional and explicit-environment tensor-network methods [15, 16]. Broader comparisons of these algorithms, software packages, and their detailed applicable regimes are already available in recent reviews [17, 18].

Then enters the hero of this paper, `ProcessTensors.jl`. BA BA BAA BAAMMMMMM. We are so easy to use. We also have a lot of documentation and examples. Someone just after learning basic Julia and reading this paper, can already install and use our software to fiddle with process tensors. We are the best. Just Kidding. The remaining part of the intro will be completed once we finalise the main contents of this scipost paper. Stay tuned!

2 Liouville-space and tensor-network foundations

The process tensor is a map acting on other maps. Before constructing such an object, it is useful to place density matrices and quantum operations on equal footing. Moving to the Liouville space does precisely this: density matrices become state-like vectors, while operations on density matrices become ordinary linear operators. This change of language is especially natural for tensor networks, because the same ideas can be read in three equivalent ways—as an equation, as a tensor diagram, and as the corresponding ITensor code. We develop these three descriptions together below. Our graphical conventions are adapted from the open-system tensor calculus of Ref. [19], with time and contractions drawn from right to left to match the convention used throughout this paper. In case the readers are not familiar with ITensors and their notations of sites and indices, they are encouraged to read the documentation of the ITensor library [[cite ITensor docs, and also the original papers](#)]. Alternatively, they can also go through the tutorials in the stable docs of our package [[add the link to the stable docs](#)].

2.1 A graphical dictionary for tensors

A tensor is represented by a shape with one open wire for every index. In our conventions, the direction of a wire records which dual space that leg belongs to: a leg pointing to the left is a ket index [Fig. 1a], whereas a leg pointing to the right is a bra index [Fig. 1b]. Joining two wires means summing over their shared index. For example, ordinary matrix multiplication,

$$(AB)_{ij} = A_{ik}B_{kj}, \quad (6)$$

is drawn by joining the right-hand wire of A to the left-hand wire of B [Fig. 1e]. Stacking two disconnected tensors vertically, with ket wires on the left and bra wires on the right, represents their tensor product [Fig. 1f]. Closing the two wires of an operator on the same index with a loop gives its trace [Fig. 1g],

$$\text{Tr}(A) = A_{ii}. \quad (7)$$

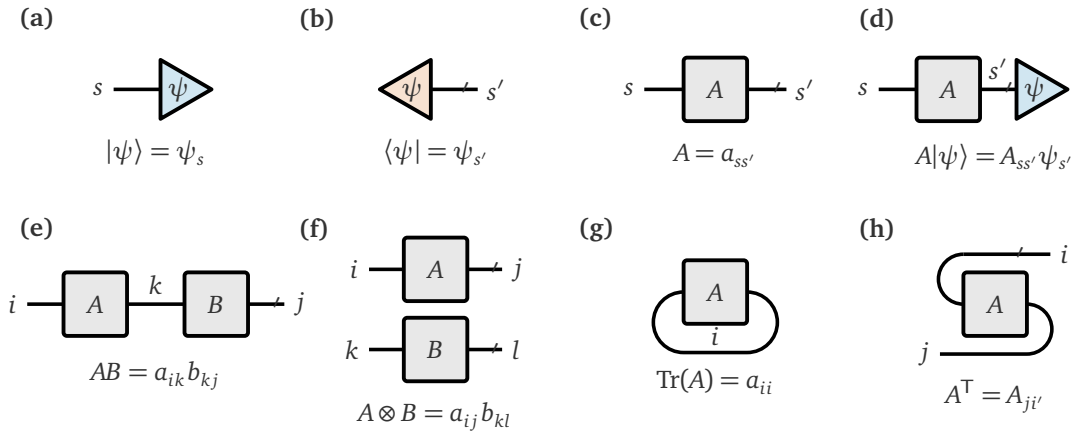


Figure 1: Graphical dictionary used throughout this work. (a) A ket $|\psi\rangle$ with unprimed index s , (b) A bra $\langle\psi|$ with primed index s' , (c) An operator A with a ket wire s on the left and a bra wire s' on the right. (d) Contracting the primed wire of A with $|\psi\rangle$. (e) The matrix multiplication $AB = a_{ik}b_{kj}$. (f) The tensor product $A \otimes B = a_{ij}b_{kl}$, drawn by stacking A above B . (g) The trace $\text{Tr}(A) = a_{ii}$. (h) The transpose $A^T = A_{ji}$, obtained by bending the left and right wires to the opposite directions.

Equations (6) and (7), Fig. 1, and Listing 1 express the same basic rule: tensors contract when their indices match. Wire bending deserves one warning though, when we implement

them in ITensor codes. A bend represents a basis-dependent transpose (swapping two indices), complex conjugation changes the tensor entries, and combining the two gives the Hermitian adjoint, $A^\dagger = (A^*)^\top$. An ITensor prime operation, by contrast, only changes the identity of an index. It does not transpose or conjugate the tensor. For achieving a dagger, we need to use the Hermitian adjoint function `dag` and then prime the indices. This distinction becomes important when we use an unprimed index for the input and a primed copy for its output when constructing operator ITensors.

Julia

```

1  using ITensors
2
3  s = Index(2, "s")
4
5  psi = ITensor([1.0, 0.0], s) # (a) ket |psi> = psi_s
6
7  psidag = dag(prime(psi)) # (b) bra <psi| = psi_s'
8
9  A = random_itensor(s, prime(s)) # (c) operator A = a_ss'
10
11 Apsi = A * prime(psi) # (d) A|psi> = A_ss' psi_s'
12
13 i = Index(2, "i")
14 k = Index(2, "k")
15 j = Index(2, "j")
16 AB = random_itensor(i, k) * random_itensor(k, j) # (e) AB = a_ik b_kj
17
18 l = Index(2, "l")
19 AxB = random_itensor(i, j) * random_itensor(k, l) # (f) A ox B = a_ij
    ↪ b_kl (no shared index)
20
21 TrA = A * delta(s, prime(s)) # (g) Tr(A) = a_ii
22
23 AT = conj(dag(A)) # (h) AT = a_ji

```

Listing 1: ITensor codes to realise tensor operations in Fig. 1(a)–(h).

2.2 From operator space to Liouville space

Let $\mathcal{H}$ be a d -dimensional Hilbert space of state vectors. Associated with it is the operator space

$$\mathcal{B}(\mathcal{H}) = \{A : \mathcal{H} \rightarrow \mathcal{H}\}, \quad (8)$$

containing all linear operators acting on $\mathcal{H}$. Density matrices, observables, Hamiltonians, and jump operators are therefore different inhabitants of the same space; the construction below is not restricted to Hermitian operators or physical states. The operator space becomes a d^2 -dimensional Hilbert space when equipped with the Hilbert–Schmidt inner product

$$\langle\langle B|A \rangle\rangle = \text{Tr}(B^\dagger A). \quad (9)$$

We refer to this operator Hilbert space as the *Liouville space*, $\mathcal{L}(\mathcal{H})$. The name reflects the historical description of dynamics directly in operator space, most notably through the quantum Liouville–von Neumann equation.

To use ordinary vector and tensor-network operations in this space, we choose a vector representation

$$A \in \mathcal{B}(\mathcal{H}) \mapsto |A\rangle\rangle \in \mathcal{L}(\mathcal{H}), \quad (10)$$

called *vectorisation*. This does not turn A into a new physical state; it represents the same operator as a vector in Liouville space by rearranging the elements into a vector. In particular, density operators, observables, and non-Hermitian operators can all be vectorised in precisely the same way.

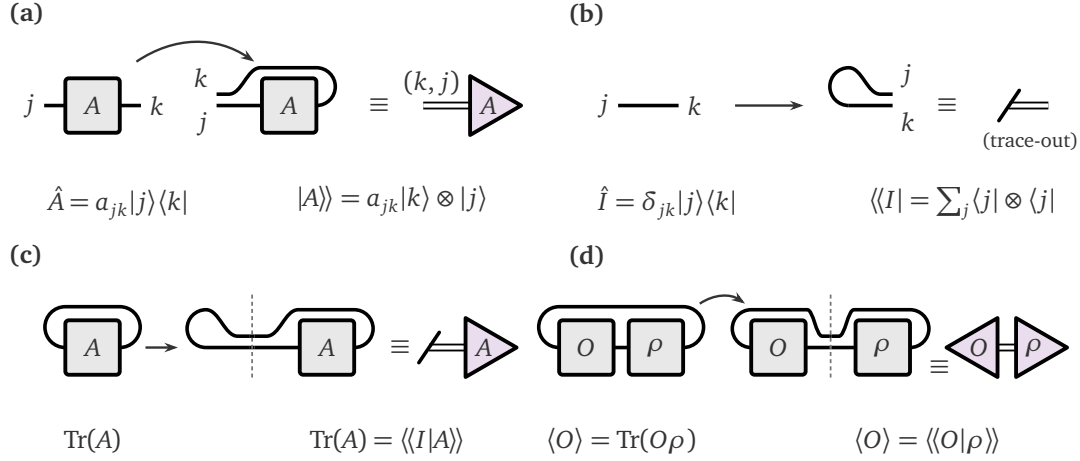


Figure 2: Vectorisation and Liouville-space closures. (a) An operator $\hat{A} = a_{jk} |j\rangle\langle k|$ is vectorised by bending the bra index k over the top, giving column-major order. The fused pair is a double Liouville wire. (b) The identity $\hat{I} = \delta_{jk} |j\rangle\langle k|$ is a straight wire. Bending the left index to the right yields the trace-out operation. (c) The Hilbert-space trace is the Liouville inner product $\text{Tr}(A) = \langle\langle I | A \rangle\rangle$. (d) An expectation value is the same contraction with an observable, $\langle O \rangle = \langle\langle O | \rho \rangle\rangle$.

Different vectorisation conventions arrange the matrix entries differently. The package follows Julia's column-major ordering. For $a_{jk} = \langle j | A | k \rangle$, we define

$$|A\rangle\rangle \equiv \text{vec}(A) = \sum_{j,k} a_{jk} |k\rangle \otimes |j\rangle, \quad (11)$$

where the first factor records the bra index k (slowest varying index) and the second the ket index j (fastest varying index), and the new double-wired index is a Liouville wire of the tuple (k, j) . This is represented as a bent wire with two indices pointing in the same direction in Fig. 2a (note that if we bend the ket index k downwards, that would correspond to a row-major order because the tuple has j as the slowest varying index, and k as the fastest varying index, i.e. the order (j, k)). Equivalently, if we look at the matrix elements of A as a 2D array, the columns of A are stacked vertically in a column-major order:

$$\begin{pmatrix} a_{11} & a_{12} \\ a_{21} & a_{22} \end{pmatrix} \mapsto \begin{pmatrix} a_{11} \\ a_{21} \\ a_{12} \\ a_{22} \end{pmatrix}. \quad (12)$$

Thus,

$$\text{vec}(|j\rangle\langle k|) = |k\rangle \otimes |j\rangle. \quad (13)$$

An important Liouville vector used repeatedly below is the vectorised identity. Graphically, it is represented as a bent wire with two indices pointing in the same direction,

$$|I\rangle\rangle = \sum_j |j\rangle \otimes |j\rangle, \quad (14)$$

as also shown in panel (b) of Fig. 2. Using Eq. (9) and substituting B with the identity operator, we see in Fig. 2(c) that the corresponding trace operation in Liouville space is expressed as an inner product:

$$\text{Tr}(A) = \langle\langle I|A\rangle\rangle. \quad (15)$$

The identity operator, I , is the maximally mixed state up to a normalisation $\rho_{\text{mm}} = I/d$. Vectorising it (panel (b)) graphically yields the operation of closing a leg, or tracing out that subsystem, as illustrated in Fig. 2(b) and (c). More generally, an observable $O = O^\dagger$ closes the same Liouville leg and returns its expectation value, as depicted in Fig. 2(d):

$$\text{Tr}(\rho) = \langle\langle I|\rho\rangle\rangle, \quad \langle O \rangle = \text{Tr}(O\rho) = \langle\langle O|\rho\rangle\rangle. \quad (16)$$

Leaving the leg unclosed, as in panel (a), instead retains the state-like object $|\rho\rangle\rangle$. These closure rules, visualised in the respective panels of Fig. 2, will reappear when process-tensor outputs are traced, measured, or left open.

Listing 2 realises the four panels of Fig. 2 with the package converters: an operator becomes a Liouville vector, the identity becomes the trace-out effect, and traces or expectation values are ordinary inner products.

Julia

```

1  using ProcessTensors
2  using ITensors: inner
3
4  sites = siteinds("S=1/2", 1)
5  sites_L = liouv_sites(sites)
6
7  A = OpSum()
8  A += 1.0, "Sx", 1
9  A_L = to_liouville(MPO(A, sites); sites=sites_L) # (a) |A>>
10
11 Id = OpSum()
12 Id += 1.0, "Id", 1
13 Id_L = to_liouville(MPO(Id, sites); sites=sites_L) # (b) <<I|
14
15 TrA = inner(Id_L, A_L) # (c) Tr(A) = <<I|A>>
16
17 rho_L = to_liouville(to_dm(MPS(sites, ["Up"]))); sites=sites_L
18 O = OpSum()
19 O += 1.0, "Sz", 1
20 O_L = to_liouville(MPO(O, sites); sites=sites_L)
21 expect_O = inner(O_L, rho_L) # (d) <O> = <<O|rho>>

```

Listing 2: Vectorisation, the identity, and Liouville closures matching Fig. 2(a)–(d).

2.3 Left and right actions

Vectorisation becomes especially useful when we want to describe an operation such as $A\rho B$ as a linear action on the Liouville-space vector $|\rho\rangle\rangle$, because it brings the operations on the bra

and ket bases to the same space. This allows Hamiltonian evolution, dissipative jumps, and process-tensor contractions to be written as ordinary tensor-network operations or MPOs. The only small surprise is that multiplication from the right does not look exactly like multiplication from the left after vectorisation. The transpose appearing below is the bookkeeping rule that keeps the two descriptions consistent.

This viewpoint is closely related to the Choi–Jamiołkowski isomorphism [20, 21]. A linear map $\Phi : \mathcal{B}(\mathcal{H}) \rightarrow \mathcal{B}(\mathcal{H})$ can be represented in several equivalent ways. It may be described by its action on operators,

$$\rho \mapsto \Phi(\rho),$$

by a Liouville-space superoperator S_Φ ,

$$|\Phi(\rho)\rangle\rangle = S_\Phi |\rho\rangle\rangle,$$

or by its Choi operator,

$$J(\Phi) = (\mathcal{I} \otimes \Phi)(|I\rangle\rangle\langle\langle I|).$$

The Liouville-space matrix and the Choi operator contain the same map, but organise its input and output indices differently. In this work, we use the Liouville-space representation because it is the form that can be applied directly to tensor-network states. The Choi representation is particularly useful when studying complete positivity and quantum channels; introductory discussions can be found in Refs. [22, 23]. **Fidel: Check if this para is sound, and see if you need more elaboration/correction.**

The connection between operator multiplication and Liouville-space action is most easily seen by sliding an operator around the same bend used to vectorise the identity in Fig. 2b. For the vectorised identity,

$$(I \otimes A)|I\rangle\rangle = (A^\top \otimes I)|I\rangle\rangle. \quad (17)$$

Graphically, A sits on the ket rail of the cup; after it slides through the bend it reappears transposed on the bra rail [Fig. 3a]. The transpose is not an additional physical operation; it records the change in index ordering induced by our column-major vectorisation convention. Applying this identity to a product gives Roth’s lemma,

$$\text{vec}(A\rho B) = (B^\top \otimes A)|\rho\rangle\rangle. \quad (18)$$

Figure 3b draws the same statement: A , ρ , and B on the ket rail are equivalent to $B^\top$ and A acting on the two legs of the bent ρ , which after connecting to Fig. 1f, gives the Liouville operator $B^\top \otimes A$ on $|\rho\rangle\rangle$. Thus the same transformation can be viewed in three equivalent ways,

$$A\rho B \longleftrightarrow \Phi_{A,B}(\rho) = A\rho B \longleftrightarrow S_{A,B} = B^\top \otimes A.$$

The first expression is ordinary operator multiplication, the second is a linear map acting on operator space, and the third is its matrix representation in Liouville space.

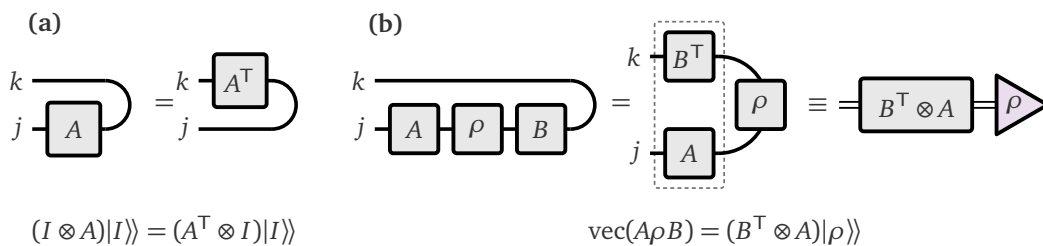


Figure 3: Wire sliding and Roth’s lemma. (a) An operator that slides through the cup emerges transposed. (b) The same move on a product yields $\text{vec}(A\rho B) = (B^\top \otimes A)|\rho\rangle\rangle$.

The two special cases used throughout the package are

$$A\rho \longleftrightarrow (I \otimes A)|\rho\rangle\rangle \equiv A_L|\rho\rangle\rangle, \quad (19)$$

$$\rho B \longleftrightarrow (B^\top \otimes I)|\rho\rangle\rangle \equiv B_R|\rho\rangle\rangle. \quad (20)$$

Setting $B = I$ in Eq. (18) leaves A on the ket rail and gives the left action A_L , likewise setting $A = I$ leaves $B^\top$ on the bra rail and gives the right action B_R . The suffixes L and R describe multiplication from the left and right of the density operator before vectorisation. Listing 3 realises the two special cases of Fig. 3b, together with the left-right jump that follows from the same identity.

Julia

```

1 using ProcessTensors
2
3 sites = siteinds("S=1/2", 1)
4 sites_L = liouv_sites(sites)
5 sL = only(sites_L)
6
7 A_L = op("Sx_L", sL)          # vec(Sx * rho)
8 B_R = op("Sz_R", sL)          # vec(rho * Sz)
9 A_jump = op("S-_Jump", sL)    # vec(S- * rho * S+)
```

Listing 3: Package notation for left, right, and left-right actions.

In this notation, Sx_L represents multiplication by S_x from the left, while Sz_R represents multiplication by S_z from the right. The $_Jump$ suffix represents the corresponding left-right action, also known as the sandwich action in the context of Local Master Equation (LME) superoperators. Together, these local operators provide the elementary building blocks for the Liouville-space dynamics and process-tensor constructions developed below.

2.4 Vectorised many-body MPS and Liouvillian MPOs

A single quantum degree of freedom lives in one Hilbert space. A many-body system brings several of them together: if site j has a local Hilbert space $\mathcal{H}_j$ of dimension d_j , then the full state belongs to the joint Hilbert space

$$\mathcal{H} = \bigotimes_{j=1}^N \mathcal{H}_j, \quad \dim \mathcal{H} = \prod_{j=1}^N d_j. \quad (21)$$

The price of adding sites is therefore immediate: the number of amplitudes grows exponentially. Tensor networks tame this growth by storing the local degrees of freedom in the form of a matrix-product state (MPS),

$$|\psi\rangle = \sum_{s_1, \dots, s_N} A_1^{s_1} A_2^{s_2} \cdots A_N^{s_N} |s_1 \cdots s_N\rangle. \quad (22)$$

Neighbouring cores are joined by virtual bonds. Their dimensions control how much correlation and entanglement the MPS can represent across each spatial cut, and truncating them provides the compression that makes the calculation practical. Standard introductions to MPS and matrix-product operators (MPOs) can be found in Refs. [24, 25]; the ITensor implementation used here is described in Ref. [26].

A mixed many-body state is described by a density operator on this joint Hilbert space,

$$\rho = \sum_{s,s'} \rho_{s,s'} |s_1 \cdots s_N\rangle \langle s'_1 \cdots s'_N|, \quad (23)$$

where $s = (s_1, \dots, s_N)$. When converted to an MPO, each local core now has two physical legs: an unprimed ket index s_j and a primed bra index s'_j , along with the virtual bond indices connecting adjacent cores.

Vectorisation of this density operator can now be performed one site at a time. As shown in Fig. 4, bending each bra wire towards its ket wire produces the column-major pair (s'_j, s_j) . A local combiner C_j then fuses this pair into the Liouville index

$$\alpha_j = (s'_j, s_j), \quad C_j : \mathcal{H}_j^* \otimes \mathcal{H}_j \longrightarrow \mathcal{L}_j, \quad \dim \mathcal{L}_j = d_j^2. \quad (24)$$

Applying these combiners across the chain turns the MPO{Hilbert} representation of ρ into an MPS{Liouville} representation of $|\rho\rangle\rangle$. The virtual bonds are untouched; only the two local physical legs are fused. Thus vectorisation changes how the same object is represented, without introducing a second physical state.

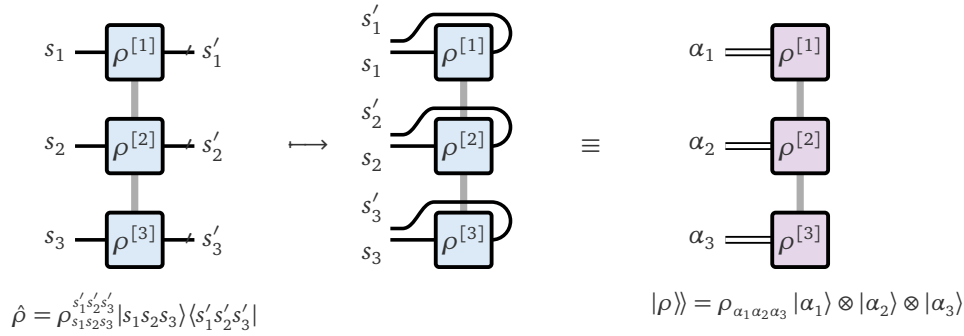


Figure 4: Local many-body vectorisation of a three-site density matrix. A Hilbert-space MPO{Hilbert} has an unprimed ket s_j and a primed bra s'_j on every site. Bending each bra over its ket gives the column-major pair (s'_j, s_j) , which is fused into the Liouville index α_j of the resulting MPS{Liouville}.

In `ProcessTensors.jl`, `to_dm` constructs a Hilbert-space density MPO and `to_liouville` applies the local fusion. The resulting Liouville MPS can then be contracted with vectorised observables or the vectorised identity, exactly as in the single-site discussion.

Julia

```

1 using ProcessTensors
2
3 sites = siteinds("S=1/2", 3)
4 sites_L = liouv_sites(sites)
5
6 psi1 = MPS(sites, ["Up", "Up", "Up"])
7 psi2 = MPS(sites, ["Dn", "Dn", "Dn"])
8 rho = to_dm([psi1, psi2]; coeffs=[0.8, 0.2])
9 rho_L = to_liouville(rho; sites=sites_L)
10
11 Id = OpSum()
12 Id += 1.0, "Id", 1
13 0 = OpSum()
```

```

14  O += 1.0, "Sz", 2
15
16  Id_L = to_liouville(MPO(Id, sites); sites=sites_L)
17  O_L = to_liouville(MPO(O, sites); sites=sites_L)
18  normalisation = inner(Id_L, rho_L)
19  expectation = inner(O_L, rho_L)

```

Listing 4: Three-site mixed state matching Fig. 4.

There is a point worth noting while defining the Liouville indices and using them to construct the corresponding Liouville-space object. The Liouville indices should be created once and reused. Calling `liouv_sites` independently produces indices with the same dimensions and tags but different identities (IDs), so `ITensor` will not contract them as the same legs. The same issue appears when one constructs a density matrix without inputting the `liouv_sites`. This would internally create a new set of indices from the Hilbert-space indices with a different index ID.

For a closed system, the density operator obeys the von Neumann equation

$$\frac{d\rho(t)}{dt} = -i[H, \rho(t)] = -i(H\rho(t) - \rho(t)H). \quad (25)$$

Using the left and right actions from Eqs. (19) and (20), this becomes one operator acting on a Liouville vector,

$$\frac{d}{dt}|\rho(t)\rangle\rangle = \mathcal{L}_H|\rho(t)\rangle\rangle, \quad \mathcal{L}_H = -i(H_L - H_R). \quad (26)$$

For an `OpSum` $H = \sum_m h_m$, the package constructs this generator term by term: every operator string h_m is converted into its `_L` and `_R` actions before the Liouville MPO is built. It does not first construct the full Hilbert-space Hamiltonian MPO and vectorise that object. This keeps the construction local and preserves the structure already present in the `OpSum`.

Going beyond the unitary time evolution, the most common Markovian open-system generator is the Gorini–Kossakowski–Lindblad–Sudarshan (GKLS) master equation,

$$\frac{d\rho}{dt} = -i[H, \rho] + \sum_{\mu} \gamma_{\mu} \left(L_{\mu} \rho L_{\mu}^{\dagger} - \frac{1}{2} L_{\mu}^{\dagger} L_{\mu} \rho - \frac{1}{2} \rho L_{\mu}^{\dagger} L_{\mu} \right). \quad (27)$$

The dictionary developed above translates it directly into

$$\mathcal{L} = -i(H_L - H_R) + \sum_{\mu} \gamma_{\mu} \left[(L_{\mu})_{\text{Jump}} - \frac{1}{2} (L_{\mu}^{\dagger} L_{\mu})_L - \frac{1}{2} (L_{\mu}^{\dagger} L_{\mu})_R \right], \quad (28)$$

where $(L_{\mu})_{\text{Jump}} = L_{\mu}^* \otimes L_{\mu}$. The package suffixes `_Jump`, `_LdagL_L`, and `_LdagL_R` implement precisely these three dissipative terms.

Julia

```

1  H = OpSum()
2  H += 0.7, "Sx", 1
3  H += 0.2, "Sz", 1
4
5  L_H = liouvillian_mpo(H, sites_L)
6  L_open = liouvillian_mpo(
7      H,
8      sites_L;

```

```

9      jump_ops=[(0.1, "S-", 1)],
10 )

```

Listing 5: Constructing Hamiltonian and dissipative Liouville MPOs.

We have arrived at a familiar place where we have described the generator of the Liouville-space dynamics in the form of a Liouville MPO. This can be readily used to perform open-quantum system dynamics, analogous to how we use the Hamiltonian MPO to perform unitary time evolution. Over a short interval Δt , the time-evolution operator is given by

$$|\rho(t + \Delta t)\rangle\rangle = e^{\Delta t \mathcal{L}} |\rho(t)\rangle\rangle. \quad (29)$$

Knowing $\mathcal{L}$, one can approximate the exponent of this MPO by using Suzuki-Trotter decomposition, as in TEBD [cite TEBD literature](#), or one can variationally optimize the time-evolved state knowing the generator using TDVP [cite TDVP literature](#) to simulate open-quantum dynamics. Appendix B gives minimal examples where we can use the existing algorithms in ITensorMPS.jl, extend it to the Liouville space, and perform open-quantum dynamics.

Table 1: Hilbert-space expressions, Liouville-space representations, and their ProcessTensors.jl counterparts.

Hilbert-space object	Liouville-space object	Package representation
$ \psi\rangle$	state vector	MPS{Hilbert}
ρ	$ \rho\rangle\rangle$	MPS{Liouville}
density operator	ket-bra legs fused locally	to_liouville
$A\rho$	$A_L \rho\rangle\rangle$	"A_L"
ρA	$A_R \rho\rangle\rangle$	"A_R"
$A\rho B$	$(B^T \otimes A) \rho\rangle\rangle$	left-right action
$A\rho A^\dagger$	$(A^* \otimes A) \rho\rangle\rangle$	"A_Jump"
$\text{Tr } \rho$	$\langle\langle I \rho\rangle\rangle$	inner(Id_L, rho_L)
$\langle O \rangle$	$\langle\langle O \rho\rangle\rangle$	inner(O_L, rho_L)
$-i[H, \rho]$	$\mathcal{L}_H \rho\rangle\rangle$	liouvillian_mpo

3 Process tensor theory and construction

3.1 From quantum channels to multi-time processes

Let us zoom out from the tensor-network details of Sec. 2 for a moment. There, density operators were vectorised, generators were written as many-body MPOs, and their action became an ordinary tensor contraction. Beneath this machinery lies a simple physical requirement: a state evolving from one time to another must remain a valid quantum state. In particular, it must remain Hermitian and positive so that measurement probabilities are real and non-negative, and it must retain unit trace so that those probabilities add up to one. A quantum channel is the linear map that enforces these requirements. Writing

$$\rho' = \Phi[\rho], \quad \Phi : \mathcal{B}(\mathcal{H}_{\text{in}}) \longrightarrow \mathcal{B}(\mathcal{H}_{\text{out}}), \quad (30)$$

the map Φ is required to be completely positive and trace preserving (CPTP),

$$(\Phi \otimes \mathcal{I}_A)[X] \geq 0 \quad \text{for every } X \geq 0, \quad (31)$$

$$\text{Tr}(\Phi[\rho]) = \text{Tr}(\rho). \quad (32)$$

The word *complete* is important. Positivity must be preserved not only for an isolated system, but also when the map acts on one part of a larger state entangled with an otherwise untouched ancilla A . Thus, behind the formal acronym “CPTP” are the familiar probability rules of quantum mechanics, together with the demand that they remain consistent when the system is embedded in a larger experiment [8].

We have already encountered one such example, although we did not call it a channel at the time. In Sec. 2, Markovian open-system dynamics was generated by the Gorini–Kossakowski–Lindblad–Sudarshan (GKLS) equation

$$\frac{d\rho}{dt} = \mathcal{L}_t[\rho] = -i[H(t), \rho] + \sum_{\mu} \gamma_{\mu}(t) \left(L_{\mu} \rho L_{\mu}^{\dagger} - \frac{1}{2} \{ L_{\mu}^{\dagger} L_{\mu}, \rho \} \right), \quad \gamma_{\mu}(t) \geq 0. \quad (33)$$

Strictly speaking, the Liouvillian $\mathcal{L}_t$ is the generator rather than the channel itself. The corresponding propagator

$$\Phi_{t:s} = \overleftarrow{\mathcal{T}} \exp \left[\int_s^t d\tau \mathcal{L}_{\tau} \right], \quad \rho(t) = \Phi_{t:s}[\rho(s)], \quad (34)$$

is CPTP; for a time-independent generator this reduces to $\Phi_{t:s} = \exp[(t-s)\mathcal{L}]$. The Liouvillian MPO introduced previously is therefore the tensor-network representation of the generator whose propagation carries one density operator to another without leaving the space of physical states.

A second, and more general, route to a quantum channel is to regard the apparent non-unitarity of the system as part of a larger unitary evolution. It is straightforward to see that a unitary transformation on a many-body density operator is a CPTP map: they preserve the state norm by definition, $U\rho U^{\dagger}$ is Hermitian if ρ is Hermitian, and it keeps the same eigenvalues under the transformation, ensuring positivity. If the system begins uncorrelated with an environment in a fixed state ρ_E , then

$$\Phi_{t:s}[\rho_S] = \text{Tr}_E [U_{t:s}(\rho_S \otimes \rho_E)U_{t:s}^{\dagger}] = \sum_{\mu} K_{\mu} \rho_S K_{\mu}^{\dagger}, \quad (35)$$

where the Kraus operators obey

$$\sum_{\mu} K_{\mu}^{\dagger} K_{\mu} = I_S. \quad (36)$$

In finite dimensions, a linear map is CPTP if and only if it admits such a Kraus representation; equivalently, every quantum channel can be realised by adjoining an initially uncorrelated environment, evolving the joint state unitarily, and discarding the environment afterwards [8]. Unitary evolution and GKLS propagation are therefore not separate ideas, but familiar members of the same family of physical state-to-state maps.

All these channels answer essentially the same question: given the state of the system now, what will its state be later? For a memoryless evolution, this state-to-state picture may be applied successively,

$$\rho_S(t_{k+1}) = \Phi_{k+1:k}[\rho_S(t_k)]. \quad (37)$$

But must the map $\Phi_{k+1:k}$ be determined by the present reduced state alone? Suppose two experimental histories produce exactly the same $\rho_S(t_k)$, while leaving behind different correlations between the system and its environment. An operation performed at an earlier time may have disturbed the environment, conditioned it on a measurement outcome, or stored information in degrees of freedom that are no longer visible in $\rho_S(t_k)$. The same current system state can then have different futures. In that situation, the question “what state comes next?” has omitted part of the experimental input: “how did we reach the present state?”

Figure placeholder: from a channel to a multi-time process

Panel (a): a single CPTP map $\Phi_{t:s}$ taking $\rho_S(s)$ to $\rho_S(t)$.

Panel (b): joint system–environment propagation interrupted by $\mathcal{A}_0, \dots, \mathcal{A}_{n-1}$, with the environment carrying information between the intervention times.

Panel (c): the same microscopic network enclosed inside $\mathcal{T}_{n:0}$, leaving one open slot for each $\mathcal{A}_k$ and a final system output. Time should run from right to left, consistently with the remaining tensor diagrams.

Figure 5: A quantum channel describes one transformation from an input state to an output state. A process tensor retains the underlying system–environment evolution while leaving the intermediate system operations open, thereby describing an entire multi-time experiment.

The natural remedy is to promote the history of interventions to an input of the dynamical description. Let $\mathcal{A}_k$ denote the operation applied to the system at time t_k , while the joint system–environment dynamics between consecutive times is described by $\mathcal{U}_{k+1:k}$. Starting from a possibly correlated state $\rho_{SE}(t_0)$, the final reduced state is

$$\begin{aligned}\rho_S(t_n) &= \text{Tr}_E[\mathcal{U}_{n:n-1}(\mathcal{A}_{n-1} \otimes \mathcal{I}_E) \cdots \mathcal{U}_{1:0}(\mathcal{A}_0 \otimes \mathcal{I}_E)[\rho_{SE}(t_0)]] \\ &=: \mathcal{T}_{n:0}[\mathcal{A}_{n-1}, \dots, \mathcal{A}_0].\end{aligned}\tag{38}$$

The process tensor $\mathcal{T}_{n:0}$ is the higher-order map that remains after the initial joint state, the inaccessible environment, and all intermediate system–environment propagators have been absorbed into a single object [6–8]. A channel accepts a state and returns a state; a one-slot superchannel accepts an operation and returns a state or another operation. A process tensor (also referred to as quantum combs, commonly in the literature) is their multi-time extension: it accepts an ordered sequence of operations and returns the final state. If the final output is also measured or traced, the same object instead returns the associated probability or expectation value.

Despite acting on operations rather than directly on states, a physical process tensor inherits three familiar structural properties from quantum mechanics [6–8]. First, it is *linear in every intervention*. If an operation at time t_k is chosen as a probabilistic mixture $p\mathcal{A}_k + (1-p)\mathcal{B}_k$, then the resulting output is the same mixture of the outputs obtained from $\mathcal{A}_k$ and $\mathcal{B}_k$. For example,

$$\begin{aligned}\mathcal{T}_{n:0}[\dots, p\mathcal{A}_k + (1-p)\mathcal{B}_k, \dots] \\ = p \mathcal{T}_{n:0}[\dots, \mathcal{A}_k, \dots] + (1-p) \mathcal{T}_{n:0}[\dots, \mathcal{B}_k, \dots].\end{aligned}\tag{39}$$

Second, the process tensor is *completely positive*. Inserting completely positive operations at its open times must produce a positive final state, even when those operations act jointly on the system and an arbitrary untouched ancilla. This is the multi-time counterpart of Eq. (31). It guarantees that the process remains physically meaningful when it is embedded inside a larger circuit, rather than only for the isolated sequence used to define it.

Finally, the process tensor is *causal*. A choice made at a later time cannot alter statistics already recorded at an earlier time. Operationally, once the future slots are filled with deterministic trace-preserving operations and their outputs are discarded, the probabilities assigned to the earlier interventions cannot depend on which future operations were chosen. This no-signalling-from-the-future condition also supplies the appropriate multi-time notion

of normalisation: every deterministic sequence must preserve total probability. In the Choi representation it becomes a hierarchy of partial-trace constraints, which we introduce in the following subsection. Together, linearity, complete positivity, and causality distinguish a physical time-ordered process from an arbitrary multilinear tensor with matching dimensions.

A point worth emphasising is that an individual trajectory selected from a process tensor need not be deterministic. An operation inserted at an intermediate time may represent a probabilistic event, which would then, reduce the trace of the outcome quantum state. Unit trace is determined by the choice of instruments, and so, trace-preserving trajectory is not a requirement for a process tensor. We will see more on this in section 4 (Quantum instruments) **TODO: add link.**

3.2 Choi representation, instruments, and process contraction

3.3 Anatomy of a ProcessTensor

The Choi representation introduced above is complete, but its size grows exponentially with the number of intervention times. This is the temporal counterpart of the many-body problem encountered in Sec. 2: writing down every possible history as one dense tensor quickly becomes impractical. The same matrix-product architecture that compressed spatial many-body states can instead be turned along the time direction. Splitting the temporal indices into consecutive groups gives

$$[\mathcal{T}_{n:0}]_{o,i} = \sum_{\alpha_1, \dots, \alpha_{n-1}} Q_{\alpha_{n-1}}^{[n-1], o_{n-1} i_{n-1}} Q_{\alpha_{n-2} \alpha_{n-1}}^{[n-2], o_{n-2} i_{n-2}} \dots Q_{\alpha_1}^{[0], o_0 i_0}. \quad (40)$$

Here i_k and o_k are the system input and output indices at time step k , while α_k is an internal memory index joining neighbouring cores. The dimensions $\chi_k = \dim(\alpha_k)$ quantify how large an effective space must be carried across each temporal cut. A Markovian process factorises across time and needs only trivial memory links, whereas a non-Markovian process generally has $\chi_k > 1$. A small value of χ_k does not imply the absence of memory; it means that the relevant memory is compressible [6, 9].

Equation (40) is stored by `ProcessTensors.jl` as a Liouville-space MPO whose sites are ordered in time. An interior core has two temporal legs and two memory links. The input leg is primed, with prime level 1, and the output leg is unprimed, with prime level 0. Both have dimension d_S^2 , retain the physical metadata of the system Liouville site, and carry a `tstep=k` tag. The memory links carry the tags `PT`, `Link`, and their temporal position. The first and last bath links are absent from the final MPO because they have already been contracted with the initial bath state $|\rho_E(0)\rangle$ and the final trace $\langle\langle I_E |$, respectively.

The package wraps this MPO together with the physical information used to construct it. Concretely, a `ProcessTensor` stores the following fields:

Field	Meaning
<code>core</code>	The time-ordered MPO{ <code>Liouville</code> } containing the tensors $Q^{[k]}$.
<code>system</code>	The system Hamiltonian, optional Lindblad channels, and canonical Liouville site indices.
<code>environment</code>	The bath modes, their initial states and Hamiltonians, and the system–bath couplings. It is <code>nothing</code> for a bath-free process.
<code>dt, nsteps</code>	The time spacing and number of temporal MPO cores. The represented final time is $t_f = n_{\text{steps}} \Delta t$.
<code>coupling_site</code>	The canonical system Liouville index whose copies become the input and output legs of every PT core.

Figure placeholder: anatomy of the temporal MPO

Draw four rounded PT cores with time running from right to left. Each core $Q^{[k]}$ has a lower primed input leg i_k , an upper unprimed output leg o_k , and thick horizontal memory links α_k . Label the physical legs by their ITensor data: dimension d_S^2 , prime level, and `tstep=k`. Label the internal legs PT, Link, `tstep=k`. At the two boundaries, show $|\rho_E(0)\rangle\rangle$ and $\langle\langle I_E|$ being absorbed into the first and last cores.

Figure 6: Matrix-product representation of a process tensor. Open system legs mark the times at which operations may be inserted, while the horizontal bonds transmit the compressed influence of the environment between different parts of the experiment.

Unknown property access is delegated to `core`, so familiar tensor-network inspection functions can be applied directly to `pt`. For example, `linkdims(pt)` and `maxlinkdim(pt)` inspect its temporal memory bonds, while `input_sites`, `output_sites`, and `coupling_times` return the exact ITensor indices needed at particular times. Reusing these returned indices is essential: ITensor contractions match index identity, not merely equal dimensions or similar-looking tags.

3.4 Ingredients of a process-tensor construction in Julia

We now construct the smallest process that can carry non-trivial memory: one system spin coupled to one bath spin. Keeping the example deliberately small lets the dense builder retain the complete bath Liouville space, so every object printed by the package can be interpreted directly. We begin with the imports

```
1 using ITensors
2 using ProcessTensors
3 using ITensors.Ops: Exact, Trotter
```

Julia

Listing 6: Imports used throughout the dense process-tensor example.

and introduce the required ingredients one at a time.
The first ingredient is the accessible system. Let

$$H_S = hS_S^x, \quad h = 0.7. \quad (41)$$

It is constructed from an ordinary Hilbert-space spin index:

```
1 system_sites = siteinds("S=1/2", 1)
2
3 h = 0.7
4 H_S = OpSum()
5 H_S += h, "Sx", 1
6
7 system = spin_system(system_sites, H_S)
8 println(system)
```

Julia

Listing 7: System Hamiltonian and SpinSystem constructor.

which prints

Terminal

```
ProcessTensors.SpinSystem
sites: 1
space: Liouville
site dims: 4
dissipative: false
```

The first line identifies the model type. Although we supplied a Hilbert index of dimension 2, `spin_system` converted it internally into a Liouville index of dimension $2^2 = 4$; this accounts for the next three lines. The final line reports that no Lindblad jump operators were supplied. The present process-tensor builder requires a single-site system because the free-system propagation is embedded into every temporal core. An initial system state is intentionally absent: it is an experimental preparation and will later be inserted into the first open process-tensor leg as an instrument.

The inaccessible environment requires more information because it is absorbed into the process tensor rather than supplied later. For each bath mode we need its Liouville site, local Hamiltonian, initial state, and coupling to the system. We take

$$H_E = \frac{\omega_E}{2} S_E^z, \quad H_{SE} = g S_E^z S_S^z, \quad \rho_E(0) = |\Downarrow\rangle\langle\Downarrow|, \quad (42)$$

with $\omega_E = 1.1$ and $g = 0.35$:

Julia

```
1 bath_sites = siteinds("S=1/2", 1)
2 bath_sites_L = liouv_sites(bath_sites)
3
4 omega_E = 1.1
5 H_E = OpSum()
6 H_E += (omega_E / 2), "Sz", 1
7
8 psi_E0 = MPS(bath_sites, ["Dn"])
9 rho_E0 = to_dm(psi_E0)
10 rho_E0_L = to_liouville(rho_E0; sites=bath_sites_L)
11
12 g = 0.35
13 H_SE = OpSum()
14 H_SE += g, "Sz", 1, "Sz", 2
15
16 mode = spin_mode(
17     bath_sites_L,
18     H_E,
19     rho_E0_L;
20     coupling=H_SE,
21 )
22 environment = spin_bath([mode])
23
24 println(environment)
```

Listing 8: Bath Hamiltonian, initial state, coupling, and SpinBath constructor.

The two site labels in H_{SE} follow a local convention used by every independent bath mode: site 1 is the bath mode and site 2 is the system coupling site. They do not refer to two sites inside the system. The call above prints

```
ProcessTensors.SpinBath
  modes: 1
  space: Liouville
  site dims: 4
  bath Liouville dimension: 4

mode summary:
  [1] SpinMode(dim=4, H_terms=1, coupling_terms=1)
```

This output first identifies a spin bath containing one mode. Its single Liouville site again has dimension 4, so the complete bath Liouville space also has dimension $D_E = 4$. The mode summary confirms that both the local bath Hamiltonian and the mode–system coupling contain one OpSum term. Notice that $\rho_E(0)$ was vectorised on the exact indices stored in `bath_sites_L`. Two Liouville indices with the same dimension and tags are not interchangeable if they have different ITensor identities.

The last physical ingredient is the time grid,

$$t_k = k\Delta t, \quad k = 0, \dots, n_{\text{steps}}, \quad (43)$$

which is specified by

```
1 dt = 0.05
2 nsteps = 5
3 println("time grid: dt=$dt, nsteps=$nsteps, t_final=$(dt * nsteps)")
```

Listing 9: Time grid for the dense process-tensor example.

and produces

```
time grid: dt=0.05, nsteps=5, t_final=0.25
```

The builder returns five PT-MPO cores, one for each interval of duration Δt . Their time tags run from `tstep=0` to `tstep=4`; the terminal boundary lies at $t_5 = 0.25$.

3.5 Building and inspecting the process tensor

For this first construction we use the `Dense()` backend. It forms the one-step joint Liouville propagator on the full system–bath space and reshapes the bath input and output indices into adjacent temporal memory links. Suppressing the free-system splitting for a moment, a single core has the schematic form

$$Q_{\alpha_{k+1}\alpha_k}^{[k],o_k i_k} = \langle\langle \alpha_{k+1} | \exp(\Delta t \mathcal{L}_{SE}) | \alpha_k \rangle\rangle, \quad (44)$$

where α_k runs over a basis of the bath Liouville space. The initial boundary is contracted with $|\rho_E(0)\rangle\rangle$, the final boundary with $\langle\langle I_E|$, and the intermediate bath legs are identified between

consecutive cores. For a single bath of Hilbert dimension d_E , the uncompressed memory link can therefore have dimension d_E^2 . This direct construction is transparent and numerically exact for the chosen time-step propagators, but its joint matrix dimension grows as $d_S^2 d_E^2$; it is consequently intended for no bath, one mode, or small multimode reference calculations.

The complete call for our example is

Julia

```

1 pt = build_process_tensor(
2     system;
3     method=Dense(),
4     environment=environment,
5     dt=dt,
6     nsteps=nsteps,
7     alg=Exact(),
8     sys_alg=Trotter{2}(),
9     progress=false,
10    verbose=true,
11 )

```

Listing 10: Dense construction of the example process tensor.

The keyword `method=Dense()` selects the process-tensor construction backend. By contrast, `alg=Exact()` specifies how the joint bath-plus-coupling Liouville propagator is exponentiated. The system's free one-step map is then embedded into every bath core. The symmetric choice `sys_alg=Trotter{2}()` places half of this free-system map on either side of the bath core,

$$M_S(\Delta t/2) Q_{E+SE}(\Delta t) M_S(\Delta t/2), \quad (45)$$

whereas the default `Trotter{1}()` uses the asymmetric ordering $Q_{E+SE}(\Delta t) M_S(\Delta t)$. Thus `Dense()` and `Exact()` answer different questions: the former chooses how the multi-time object is assembled, while the latter chooses how one local propagator is computed.

We have disabled the transient progress animation and enabled persistent logging so that the stages remain visible after the calculation. Omitting terminal-dependent decorations, the output reads

Terminal

```

Info: Building process tensor
  method = Dense
  nsteps = 5
  dt = 0.05
  environment = SpinBath(nmodes=1, D_bath=4, coupling=true)

Info: Constructing joint propagator
  joint_dimension = 16
  channel = liouvillian

Info: Assembling process-tensor cores
  total = 5

Info: Closing bath boundaries

Info: Built process tensor
  maxlinkdim = 4

```

```
elapsed_seconds = ...
```

The opening record repeats the selected backend, temporal grid, and bath summary. The joint dimension $16 = 4 \times 4$ is the product of the system and bath Liouville dimensions; this is the matrix space on which the dense exponential is constructed. The next stage places one copy of that propagator into each of the five temporal slabs. “Closing bath boundaries” means contracting the first bath link with $\rho_E(0)$ and the last with the trace effect. Finally, `maxlinkdim=4` reports the largest surviving temporal memory space. The elapsed time is deliberately machine dependent.

The returned object has its own compact summary:

Julia

```
1 println(pt)
```

Listing 11: Printed summary of the constructed process tensor.

Terminal

```
5-step ProcessTensor{SpinSystem, SpinBath} | dt=0.05 | t_final=0.25 |
→ maxlinkdim=4

system:      SpinSystem(nsites=1, dissipative=false)
environment: SpinBath(nmodes=1, D_bath=4, coupling=true)
core:        MPO{Liouville}(length=5, linkdims=[4, 4, 4, 4])
```

The header confirms the number of temporal cores, their spacing, the represented final time, and the largest memory bond. The system and environment lines record the physical objects absorbed into the construction. The final line exposes the tensor-network anatomy: five Liouville MPO cores joined by four memory links, each of dimension 4. No temporal compression has been performed by `Dense()`, so these links retain the complete one-spin bath Liouville space.

We can inspect the open system legs without relying on the randomly generated textual IDs of individual ITensor indices:

Julia

```
1 in0 = only(input_sites(pt, 0))
2 out0 = only(output_sites(pt, 0))
3 tin = tag_value(in0, "tstep=")
4 tout = tag_value(out0, "tstep=")
5
6 println("input : dim=$(dim(in0)), plev=$(plev(in0)), tstep=$tin")
7 println("output: dim=$(dim(out0)), plev=$(plev(out0)), tstep=$tout")
8 println("memory bond dimensions: ", linkdims(pt))
9 println("largest memory bond: ", maxlinkdim(pt))
10
11 out_prev, in_curr = coupling_times(pt, 1)
12 tprev = tag_value(only(out_prev), "tstep=")
13 tcurr = tag_value(only(in_curr), "tstep=")
14 println("step 1 connects output tstep=$tprev to input tstep=$tcurr")
```

Figure placeholder: dense construction in equation, diagram, and code

Panel (a): the one-spin system, one-spin bath, their initial data, and H_{SE} . Panel (b): five copies of the dense joint propagator, with bath legs connected horizontally, the first bath leg closed by $\rho_E(0)$, and the last by $\langle\langle I_E |$. Panel (c): the resulting ProcessTensor summary and the corresponding `build_process_tensor` call.
Keep the same index colors and tags used in Fig. 6.

Figure 7: Dense construction of a process tensor from its microscopic ingredients. The inaccessible bath is closed at the temporal boundaries but retained between time steps as the MPO memory bond; the system legs remain open for later interventions.

Listing 12: Inspection of open system legs and temporal memory bonds.

The resulting output is

```
input : dim=4, plev=1, tstep=0
output: dim=4, plev=0, tstep=0
memory bond dimensions: [4, 4, 4, 4]
largest memory bond: 4
step 1 connects output tstep=0 to input tstep=1
```

Terminal

The equal physical dimensions show that both legs represent the same system Liouville space; their prime levels distinguish the state entering a temporal slab from the state leaving it. The `tstep` tags locate those legs within the process. Finally, `coupling_times(pt, 1)` returns the output at step 0 and the input at step 1, which are precisely the two legs on which an intermediate two-leg operation will act.

At this stage the process has been constructed and inspected, but no experimental question has yet been asked of it. The open legs in Fig. 7 are deliberately empty. In the next section we develop the quantum instruments that may occupy those slots; only afterwards will we assemble them into complete multi-time experiments and contract them with the process tensor.

4 Quantum instruments

4.1 Operational meaning

4.2 Instrument organisation in `ProcessTensors.jl`

4.3 `InstrumentSeq`: a temporal analogue of `OpSum`

5 Contracting and evaluating processes

5.1 General contraction with `evaluate_process`

5.2 Reduced dynamics with `evolve`

5.3 Open process legs

6 Common process-tensor workflows

6.1 Final reduced density matrices

6.2 Observable expectation values

6.3 Probabilities and trace-non-preserving branches

6.4 Multi-time correlation functions

7 Testers and enlarged interventions

Status: planned for the paper release; API under development.

8 Characterising process tensors and quantum memory

Section lead: Aaron Sander. The text below is a structural placeholder and should not be read as claiming currently released package functionality.

9 Validation and ACE benchmarks

9.1 Correctness against dense and exact references

9.2 Compression accuracy and temporal bond dimension

9.3 Runtime and memory scaling

9.4 Comparison with the C++ ACE toolkit

10 Conclusion and outlook

You must include a conclusion.

Acknowledgements

Acknowledgements should follow immediately after the conclusion.

Disclosure of generative AI use The authors disclose that the following generative AI tools have been used in the preparation of this submission. Large language models accessed through coding assistants (including Cursor and ChatGPT) were used to draft TikZ figures, to generate example code listings that were subsequently verified by hand, and to assist in the construction of `ProcessTensors.jl` package. All AI-generated source, including figure code, listing code, and package code, was examined thoroughly by the authors, tested against the intended mathematical and numerical behaviour, and is not left in an unverified “vibe-coded” state. The authors take full responsibility for the contents of the manuscript and of the software.

Author contributions This is optional. If desired, contributions should be succinctly described in a single short paragraph, using author initials.

Funding information Authors are required to provide funding information, including relevant agencies and grant numbers with linked author’s initials. Correctly-provided data will be linked to funders listed in the [Fundref registry](#).

Code and data availability The development repository is available at <https://github.com/Gauthameshwar/ProcessTensors.jl> and the documentation at <https://gauthameshwar.github.io/ProcessTensors.jl/stable/>. The final paper will cite an immutable release and archive the scripts/data used for all benchmark figures. [TODO: insert release DOI/Zenodo or SciPost codebase DOI at submission.]

A How to read the Julia listings

Code examples throughout this paper are displayed in framed Julia listings. A blue **Julia** badge in the top-right corner identifies the language. Printed program output is shown in a matching gray box with a **Terminal** badge; that text is not syntax-highlighted. Line numbers on the left are the ones referred to in the text (for example, Listing 13, lines 4–6). Syntax highlighting is chosen to make the origin of each identifier visible at a glance, rather than to reproduce a particular editor theme.

The colour roles are as follows.

- **Blue, bold:** Julia keywords such as `using`.
- **Violet, bold:** names exported by `ProcessTensors.jl`. This includes package-specific constructors and converters (`to_dm`, `liouv_sites`, `liouvillian_mpo`, ...) as well as the tensor-network types that form the package’s public surface, notably `MPS`, `MPO`, and `OpSum`.
- **Teal, bold:** functions that belong to `ITensors.jl` and are *not* part of the `ProcessTensors.jl` export list (`Index`, `ITensor`, `siteinds`, `dag`, `prime`, `delta`, `inner`, `op`, `apply`, ...).
- **Orange, bold:** package types and backends such as `Hilbert`, `Liouville`, `Dense`, and `ACE`.
- **Near-black:** ordinary variables.

- **Green, bold:** numeric literals.
- **Crimson:** string literals.
- **Grey italic:** comments.

Listing 13 collects these conventions in one place. Violet names are the objects a user of `ProcessTensors.jl` is expected to call; teal names are the underlying `ITensor` primitives on which those objects are built.

Julia

```

1  using ITensors, ProcessTensors
2
3  s = Index(2, "s")                # ITensor index
4  psi = ITensor([1.0, 0.0], s)
5  sites = siteinds("S=1/2", 1)
6
7  rho = to_dm(MPS(sites, ["Up"]))  # ProcessTensors converter +
    ↪ MPS
8  H = OpSum()
9  H += 0.7, "Sx", 1
10 L = liouvillian_mpo(H, liouv_sites(sites))

```

Listing 13: Colour legend for the listings in this paper.

B Time dynamics in Liouville space

The Liouville-space MPS and MPO objects introduced in Sec. 2.4 provide a direct representation of time-dependent open-system states and generators. If $|\rho(0)\rangle\rangle$ is an initial Liouville MPS and $\mathcal{L}$ is its Liouvillian, the state at time t is

$$|\rho(t)\rangle\rangle = \exp(t\mathcal{L})|\rho(0)\rangle\rangle. \quad (\text{B.1})$$

`ProcessTensors.jl` extends the time-evolution algorithms provided by `ITensorMPS.jl` to its typed Hilbert- and Liouville-space objects. These routines are useful for time-local master equations and provide reference trajectories against which process-tensor calculations can be validated.

For a small two-spin example, Liouville-space TEBD approximates the propagator over successive time steps by constructing local gates for $\exp(\Delta t \mathcal{L})$ from a Hilbert-space Hamiltonian and local dissipators:

Julia

```

1  using ITensors, ProcessTensors
2  using ITensors.Ops: Trotter
3
4  sites = siteinds("S=1/2", 2)
5  sites_L = liouv_sites(sites)
6  psi0 = MPS(sites, ["Up", "Dn"])
7  rho_L0 = to_liouville(to_dm(psi0); sites=sites_L)
8
9  H = OpSum()

```

```

10 H += 1.0, "Sz", 1, "Sz", 2
11 H += 0.4, "Sx", 1
12 H += 0.4, "Sx", 2
13 jumps = [(0.15, "S-", 1), (0.15, "S-", 2)]
14
15 dt, T = 0.025, 0.1
16 rho_L_tebd = tebd(
17     rho_L0, H, dt, T;
18     jump_ops=jumps,
19     alg=Trotter{2}(),
20     maxdim=32,
21     cutoff=1e-12,
22 )

```

Listing 14: Time evolution of a dissipative Liouville-space state with TEBD.

TDVP provides a complementary time-evolution route by acting with the Liouvillian MPO itself. The evolution time supplied to TDVP is real because the Hamiltonian contribution $-i[H, \rho]$ and all dissipative terms are already contained in $\mathcal{L}$:

```

1 L = liouvillian_mpo(H, sites_L; jump_ops=jumps)
2 rho_L_tdvp = tdvp(
3     L, T, rho_L0;
4     time_step=dt,
5     nsite=2,
6     maxdim=32,
7     cutoff=1e-12,
8     outputlevel=0,
9 )

```

Julia

Listing 15: Time evolution of the same Liouville-space state with TDVP.

The two methods expose the same physical time dynamics through different tensor-network approximations: TEBD controls the local Trotter decomposition, whereas TDVP controls projection onto the evolving MPS manifold. In either case, convergence should be checked with respect to the time step, truncation cutoff, and maximum bond dimension. The resulting trajectory should also be validated by monitoring trace, Hermiticity, and positivity.

Complete discussions of Trotter order, TDVP parameters, dense validation, and time-resolved observables are available in the [unitary-dynamics](#) and [dissipative-dynamics](#) tutorials. Larger time-dependent examples, including dissipative spin chains, boundary-driven transport, and driven bosonic systems, are linked from the stable package documentation. For a more detailed overview of time evolution using MPS and MPO, refer to the review by Paeckel et al. [27].

References

- [1] V. Gorini, A. Kossakowski and E. C. G. Sudarshan, *Completely positive dynamical semigroups of N-level systems*, Journal of Mathematical Physics **17**(5), 821 (1976), doi:[10.1063/1.522979](https://doi.org/10.1063/1.522979).

- [2] G. Lindblad, *On the generators of quantum dynamical semigroups*, Communications in Mathematical Physics **48**(2), 119 (1976), doi:[10.1007/BF01608499](https://doi.org/10.1007/BF01608499).
- [3] Á. Rivas, S. F. Huelga and M. B. Plenio, *Quantum non-markovianity: Characterization, quantification and detection*, Reports on Progress in Physics **77**(9), 094001 (2014), doi:[10.1088/0034-4885/77/9/094001](https://doi.org/10.1088/0034-4885/77/9/094001).
- [4] I. de Vega and D. Alonso, *Dynamics of non-markovian open quantum systems*, Reviews of Modern Physics **89**(1), 015001 (2017), doi:[10.1103/RevModPhys.89.015001](https://doi.org/10.1103/RevModPhys.89.015001).
- [5] F. A. Pollock, C. Rodríguez-Rosario, T. Frauenheim, M. Paternostro and K. Modi, *Operational markov condition for quantum processes*, Physical Review Letters **120**(4), 040405 (2018), doi:[10.1103/PhysRevLett.120.040405](https://doi.org/10.1103/PhysRevLett.120.040405).
- [6] F. A. Pollock, C. Rodríguez-Rosario, T. Frauenheim, M. Paternostro and K. Modi, *Non-Markovian quantum processes: Complete framework and efficient characterisation*, Physical Review A **97**(1), 012127 (2018), doi:[10.1103/PhysRevA.97.012127](https://doi.org/10.1103/PhysRevA.97.012127).
- [7] S. Milz and K. Modi, *Quantum stochastic processes and quantum non-Markovian phenomena*, PRX Quantum **2**(3), 030201 (2021), doi:[10.1103/PRXQuantum.2.030201](https://doi.org/10.1103/PRXQuantum.2.030201).
- [8] P. Taranto, S. Milz, M. Murao, M. T. Quintino and K. Modi, *Higher-order quantum operations*, doi:[10.48550/arXiv.2503.09693](https://doi.org/10.48550/arXiv.2503.09693) (2025), [2503.09693](https://arxiv.org/abs/2503.09693).
- [9] M. R. Jørgensen and F. A. Pollock, *Exploiting the causal tensor network structure of quantum processes to efficiently simulate non-markovian path integrals*, Physical Review Letters **123**(24), 240602 (2019), doi:[10.1103/PhysRevLett.123.240602](https://doi.org/10.1103/PhysRevLett.123.240602).
- [10] A. Strathearn, P. Kirton, D. Kilda, J. Keeling and B. W. Lovett, *Efficient non-markovian quantum dynamics using time-evolving matrix product operators*, Nature Communications **9**, 3322 (2018), doi:[10.1038/s41467-018-05617-3](https://doi.org/10.1038/s41467-018-05617-3).
- [11] M. Cygorek, M. Cosacchi, A. Vagov, V. M. Axt, B. W. Lovett, J. Keeling and E. M. Gauger, *Simulation of open quantum systems by automated compression of arbitrary environments*, Nature Physics **18**, 662 (2022), doi:[10.1038/s41567-022-01544-9](https://doi.org/10.1038/s41567-022-01544-9).
- [12] V. Link, H.-H. Tu and W. T. Strunz, *Open quantum system dynamics from infinite tensor network contraction*, Physical Review Letters **132**(20), 200403 (2024), doi:[10.1103/PhysRevLett.132.200403](https://doi.org/10.1103/PhysRevLett.132.200403), [2307.01802](https://arxiv.org/abs/2307.01802).
- [13] G. E. Fux, P. Fowler-Wright, J. Beckles, E. P. Butler, P. R. Eastham, D. Gribben, J. Keeling, D. Kilda, P. Kirton, E. D. C. Lawrence, B. W. Lovett, E. O'Neill *et al.*, *OQuPy: A Python package to efficiently simulate non-markovian open quantum systems with process tensors*, The Journal of Chemical Physics **161**(12), 124108 (2024), doi:[10.1063/5.0225367](https://doi.org/10.1063/5.0225367).
- [14] M. Cygorek and E. M. Gauger, *ACE: A general-purpose non-markovian open quantum systems simulation toolkit based on process tensors*, The Journal of Chemical Physics **161**(7), 074111 (2024), doi:[10.1063/5.0221182](https://doi.org/10.1063/5.0221182).
- [15] A. Bose, *QuantumDynamics.jl: A modular approach to simulations of dynamics of open quantum systems*, The Journal of Chemical Physics **158**(20), 204113 (2023), doi:[10.1063/5.0151483](https://doi.org/10.1063/5.0151483).
- [16] T. Lacroix, B. Le Dé, A. Riva, A. J. Dunnett and A. W. Chin, *MPSDynamics.jl: Tensor network simulations for finite-temperature (non-markovian) open quantum system dynamics*, The Journal of Chemical Physics **161**(8), 084116 (2024), doi:[10.1063/5.0223107](https://doi.org/10.1063/5.0223107).

- [17] J. Keeling, E. M. Stoudenmire, M.-C. Bañuls and D. R. Reichman, *Process tensor approaches to non-markovian quantum dynamics*, *Physical Review X* **16**, 020502 (2026), doi:[10.1103/1ncg-11hz](https://doi.org/10.1103/1ncg-11hz).
- [18] T. Lacroix, A. Burgess, N. Lorenzoni, J. Wiercinski, K. Damezin, J. Lim, D. Tamascelli, A. W. Chin, M. Cygorek, B. W. Lovett, J. Keeling, S. F. Huelga *et al.*, *Tensor network methods for non-perturbative dynamics of open quantum systems*, doi:[10.48550/arXiv.2608.09850](https://doi.org/10.48550/arXiv.2608.09850) (2026), [2608.09850](https://arxiv.org/abs/2608.09850).
- [19] C. J. Wood, J. D. Biamonte and D. G. Cory, *Tensor networks and graphical calculus for open quantum systems*, *Quantum Information and Computation* **15**(9-10), 759 (2015), doi:[10.26421/QIC15.9-10-3](https://doi.org/10.26421/QIC15.9-10-3), [1111.6950](https://arxiv.org/abs/1111.6950).
- [20] M.-D. Choi, *Completely positive linear maps on complex matrices*, *Linear Algebra and its Applications* **10**(3), 285 (1975), doi:[10.1016/0024-3795\(75\)90075-0](https://doi.org/10.1016/0024-3795(75)90075-0).
- [21] A. Jamiołkowski, *Linear transformations which preserve trace and positive semidefiniteness of operators*, *Reports on Mathematical Physics* **3**(4), 275 (1972), doi:[10.1016/0034-4877\(72\)90011-0](https://doi.org/10.1016/0034-4877(72)90011-0).
- [22] M. A. Nielsen and I. L. Chuang, *Quantum Computation and Quantum Information*, Cambridge University Press, 10th anniversary edn., ISBN 9781107002173, doi:[10.1017/CBO9780511976667](https://doi.org/10.1017/CBO9780511976667) (2010).
- [23] J. Watrous, *The Theory of Quantum Information*, Cambridge University Press, doi:[10.1017/9781316848142](https://doi.org/10.1017/9781316848142) (2018).
- [24] U. Schollwöck, *The density-matrix renormalization group in the age of matrix product states*, *Annals of Physics* **326**(1), 96 (2011), doi:[10.1016/j.aop.2010.09.012](https://doi.org/10.1016/j.aop.2010.09.012).
- [25] R. Orús, *A practical introduction to tensor networks: Matrix product states and projected entangled pair states*, *Annals of Physics* **349**, 117 (2014), doi:[10.1016/j.aop.2014.06.013](https://doi.org/10.1016/j.aop.2014.06.013).
- [26] M. Fishman, S. R. White and E. M. Stoudenmire, *The ITensor software library for tensor network calculations*, *SciPost Physics Codebases* p. 4 (2022), doi:[10.21468/SciPostPhysCodeb.4](https://doi.org/10.21468/SciPostPhysCodeb.4).
- [27] S. Paeckel, T. Köhler, A. Swoboda, S. R. Manmana, U. Schollwöck and C. Hubig, *Time-evolution methods for matrix-product states*, *Annals of Physics* **411**, 167998 (2019), doi:[10.1016/j.aop.2019.167998](https://doi.org/10.1016/j.aop.2019.167998).